

B. Tech. Project AY 2026-27

CA 1 Presentation

Progress Report



**SYMBIOSIS INSTITUTE OF
TECHNOLOGY, PUNE**

Name of the Guide: _____

Name of the Industry Mentor: _____

Group Members:

1. Name (PRN)
2. Name (PRN)
3. Name (PRN)
4. Name (PRN)

31-07-2026

Presentation roadmap

Reframe the old pitch as a progress report.

Move in order from context to outcome.

Keep the text light and the visuals doing the heavy lifting.

Visual placeholder

Visual placeholder: a one-line flow map from introduction to references; it helps the audience see the deck structure before the detail starts.

Report window and source set

Reporting window: 26 Jun 2026 to 31 Jul 2026.

Core evidence comes from the eight blog posts and the two progress reports.

The first two blogs establish the baseline; the later six trace the recovery work.

Visual placeholder

Visual placeholder: a timeline of posts #1 to #8 with the two progress-report checkpoints; this anchors the narrative in time.

Project snapshot

Project: Generalisable Pipeline for Backporting ROCm to Obsolete AMD Hardware.

Context: FOSS United Foundation, Season of Commits.

Primary target: gfx90c, with gfx902 used as the first cross-check.

Visual placeholder

Visual placeholder: a compact project card with name, venue, role, and target devices; this gives the audience the whole case in one glance.

Scope evolution

The work started as a mentorship pitch and became a recovery program.

The key question shifted from running code to reproducing fixes.

The July reports show the scope broadening into build-system and compiler work.

Visual placeholder

Visual placeholder: a before/after scope diagram; it makes the change from idea to execution obvious.

Introduction: why this matters

ROCm is AMD's open-source GPU compute stack.

Dropped support turns usable hardware into software-stranded hardware.

The project asks whether the missing path can be rebuilt systematically.

Visual placeholder

Visual placeholder: an AMD GPU stack diagram; it helps position ROCm, HIP, LLVM, and the user-space libraries in one picture.

Problem statement: dropped support

gfx90c and gfx902 are not mainstream packaged targets.

Native driver support exists, but packaged user-space support is incomplete.

HIP kernels may work even when PyTorch and rocBLAS fail.

Visual placeholder

Visual placeholder: a failure-path sketch from driver to runtime to library; it separates kernel support from packaging support.

Problem statement: hardware versus software

The hardware is capable of basic compute.

Most blockers are packaging, metadata, and missing kernels.

A small subset of failures are genuine hardware limits.

Visual placeholder

Visual placeholder: a split map with software gaps and hardware limits; it keeps the discussion from collapsing into a single vague cause.

Problem statement: why these targets

gfx90c gives a laptop APU case.

gfx902 gives a desktop APU cross-check.

Together they test whether the method generalises across form factors.

Visual placeholder

Visual placeholder: a two-device comparison card; it makes the laptop/desktop contrast easy to remember.

Objectives: immediate

Verify native ROCm on Fedora 43.

Run HIP code on gfx90c without relying on permanent spoofing.

Classify failures in PyTorch and rocm-examples.

Visual placeholder

Visual placeholder: a simple checklist; it shows the shortest-path goals before the deeper recovery work begins.

Objectives: medium term

Recover missing gfx90c kernels and Tensile data.

Rebuild the necessary pieces from source.

Compare native, spoofed, and source-built paths side by side.

Visual placeholder

Visual placeholder: a three-lane comparison graphic; it makes the source of each improvement measurable.

Objectives: long term

Complete the gfx90c port.

Build an end-to-end CI/CD pipeline.

Turn the workflow into a reusable recovery method.

Visual placeholder

Visual placeholder: a maturity ladder; it makes the jump from one-off debugging to automation visible.

Project plan: six-month port

Finalise gfx90c support across the key ROCm paths.

Keep each recovery stage benchmarked.

Treat the port as a reusable pipeline, not a patch dump.

Visual placeholder

Visual placeholder: a six-month milestone bar; it explains the first project boundary.

Project plan: nine-month CI/CD

Automate build, validation, and regression checks.

Make the recovery workflow repeatable.

Preserve target-specific artefacts and benchmark history.

Visual placeholder

Visual placeholder: a pipeline diagram with build, test, and publish nodes; it shows why CI/CD is the natural second phase.

Project plan: next project

After FOSS United provides a SOTA RDNA 4 GPU, shift to direct upstream work.

Targets: Mojo, LLVM, ROCm, PyTorch, and vLLM.

Use the first project as the benchmarked foundation.

Visual placeholder

Visual placeholder: a roadmap arrow into upstream repositories; it ties the current port to the next research block.

Literature review: blog #1

Frames the hardware target and the motivation.

Introduces the latest ROCm on old hardware question.

Sets the original research direction.

Visual placeholder

Visual placeholder: the opening cover plus the system-spec card from the post; it re-establishes the starting point.

Literature review: blog #2

Confirms native ROCm 6.4.4 on Fedora 43.

Shows HIP works while PyTorch breaks.

Points to missing gfx90c rocBLAS kernels.

Visual placeholder

Visual placeholder: the native-install and PyTorch failure screenshots; they show the first hard split between works and fails.

Literature review: blog #3

Tests gfx900 spoofing with the HSA override flag.

Finds the path useful but not production grade.

Benchmarks expose memory-architecture mismatch.

Visual placeholder

Visual placeholder: the FP32 and memory-bandwidth charts; they quantify the spoofing trade-off better than text alone.

Literature review: blog #4

Runs first-party AMD benchmarks.

Reveals /opt/rocm packaging friction.

Moves the work from anecdote to measurement.

Visual placeholder

Visual placeholder: the first-party bandwidth chart and the /opt/rocm explanation; it shows why packaging became a research topic.

Literature review: blog #5

Builds a GPU-programming foundation.

Explains CPU and GPU differences for matmul-heavy work.

Curates the learning resources used later.

Visual placeholder

Visual placeholder: a GPU-vs-CPU architecture slide; it gives the later execution-model work its motivation.

Literature review: blog #6

Uses rocm-examples as a broad validation suite.

Turns raw failures into categories.

Separates software gaps from one hardware-limited case.

Visual placeholder

Visual placeholder: the Sankey diagram from the post; it compresses 113 tests into one failure map.

Literature review: blog #7

Rebuilds rocm-examples from source.

Discovers the missing main_gfx90c.s generation path.

Shows the compiler stack from HIP to AMDGPU machine code.

Visual placeholder

Visual placeholder: the build log plus the missing-target error; it explains why source control matters here.

Literature review: blog #8

Maps work-items, wavefronts, workgroups, and dispatches.

Connects VALU, SALU, LDS, GDS, L2, VRAM, and scratch.

Explains why simple HIP kernels survive while larger stacks do not.

Visual placeholder

Visual placeholder: the two diagrams justify the analysis.

Gap in research: no recovery workflow

Support exists in fragments.

There is no single reproducible end-to-end path.

This project fills that workflow gap.

Visual placeholder

Visual placeholder: a fragmented-fixes to unified-pipeline diagram; it makes the research gap concrete.

Gap in technology: packaged artefacts

Fedora packages miss key gfx90c assets.

rocBLAS and Tensile artefacts are incomplete.

rocm-examples fails because the right files are absent.

Visual placeholder

Visual placeholder: a packaging failure tree; it shows the difference between installed software and runnable software.

Gap in methodology: weak benchmark discipline

Community fixes are usually ad hoc.

Few reports compare native, spoofed, and rebuilt paths side by side.

The project makes those comparisons explicit.

Visual placeholder

Visual placeholder: a comparison matrix; it keeps the methodology honest.

Proposed solution: staged recovery pipeline

Start with the least invasive path.

Escalate only when a new stage adds diagnostic value.

Preserve every working stage as a benchmarked check-point.

Visual placeholder

Visual placeholder: a recovery ladder; it shows how the work progresses without jumping straight to source builds.

Proposed solution: recovery ladder

Native stack.

Spoofed stack.

Mixed-version stack.

Containerised stack.

Source-built stack.

Visual placeholder

Visual placeholder: a five-step ladder diagram; each rung corresponds to a distinct failure layer.

Proposed solution: validation principle

Every candidate fix must run the same workload set.

Success means function plus performance.

A fix without reproducibility is not enough.

Visual placeholder

Visual placeholder: a pass/fail scoreboard; it prevents the deck from drifting into vague claims.

Requirement analysis: functional

HIP execution must run on gfx90c.

PyTorch and rocBLAS must reach usable states.

rocm-examples must build and run across the target set.

Visual placeholder

Visual placeholder: a requirements checklist; it keeps the implementation bounded.

Requirement analysis: non-functional

Reproducible on Fedora 43.

Explainable failure modes.

Portable enough to transfer to gfx902.

Visual placeholder

Visual placeholder: a quality checklist; it makes the non-functional bar explicit.

Technology stack: system layer

Fedora 43 is the host distribution.

amdgpu and amd-kfd provide kernel-side support.

The targets are UMA APUs rather than discrete HBM parts.

Visual placeholder

Visual placeholder: a system stack card; it anchors the runtime environment.

Technology stack: ROCm layer

ROCm 6.4.4 is the baseline stack.

hipcc, hipblas, rocblas, rocminfo, rocm-smi, and clinfo are the core tools.

Userspace packages and kernel support are treated separately.

Visual placeholder

Visual placeholder: a ROCm component map; it shows which layer each tool belongs to.

Technology stack: compiler layer

LLVM underpins HIP compilation.

hipcc lowers HIP through clang and the AMDGPU backend.

Source builds show where support ends and packaging begins.

Visual placeholder

Visual placeholder: a compiler pipeline diagram; it explains the whole translation path at a glance.

Technology stack: libraries

rocBLAS and Tensile handle matrix kernels.

MIOpen and RCCL are adjacent ROCm components.

rocm-examples serves as the validation corpus.

Visual placeholder

Visual placeholder: a library dependency web; it clarifies why one missing file can block many workloads.

Technology stack: external tooling

DNF for package inspection.

Git and CMake for source builds.

clpeak, Geekbench, and device-bandwidth tests for measurement.

Visual placeholder

Visual placeholder: a toolbelt strip; it gives the audience the practical workflow in one row.

Design: overall architecture

Measure the baseline.

Classify the failure.

Apply the next least invasive recovery step.

Re-measure before moving on.

Visual placeholder

Visual placeholder: a closed-loop architecture diagram; it shows the project as an iterative system, not a one-shot fix.

Design: decision matrix

Packaging issue -> rebuild or repack.

Metadata issue -> correct target settings.

Missing artefact -> generate or backport.

Hardware limit -> document and stop.

Visual placeholder

Visual placeholder: a decision matrix; it turns debugging into a repeatable rule set.

Design: target-generalisation plan

Validate first on gfx90c.

Re-run the surviving steps on gfx902.

Keep only the steps that transfer cleanly.

Visual placeholder

Visual placeholder: a target-transfer arrow; it shows why gfx902 is the generalisation checkpoint.

Design: evidence layout

One slide per stage.

One visual per stage where possible.

One benchmark anchor per stage.

Visual placeholder

Visual placeholder: a slide template mock-up; it keeps the later visuals consistent and easy to scan.

Development: native ROCm on Fedora 43

System ROCm 6.4.4 installs cleanly.

hipcc can execute a simple kernel.

Darktable and rocminfo confirm basic functionality.

Visual placeholder

Visual placeholder: the native install output and rocminfo screenshot; it proves the base system is not the problem.

Development: PyTorch baseline

PyTorch installs its own ROCm userspace.

gfx90c rocBLAS kernels are missing.

The packaged path breaks before useful compute.

Visual placeholder

Visual placeholder: the PyTorch install and library listing; it shows the mismatch between package contents and target needs.

Development: spoofing path

HSA_OVERRIDE_GFX_VERSION=9.0.0 enables a gfx900-like path.

It validates compatibility but not true hardware parity.

The memory and launch behaviour differ in measurable ways.

Visual placeholder

Visual placeholder: a spoofed-versus-native comparison slide; it makes the trade-off visible instead of assumed.

Development: benchmark suite

clpeak measures raw device capability.

Geekbench and CLBlast add workload-style checks.

First-party AMD bandwidth tests anchor the comparison.

Visual placeholder

Visual placeholder: a benchmark wheel or grid; it keeps the test surface broad but readable.

Development: first benchmark findings

FP32 compute is close between spoofed and native.

Memory bandwidth is also close in the headline numbers.

Latency and pinned-memory behaviour still diverge.

Visual placeholder

Visual placeholder: the FP32 and bandwidth bar charts; they compress the main performance claim into two visuals.

Development: rocm-examples as a corpus

113 examples provide a broad test surface.

The failures expose packaging and kernel gaps quickly.

The corpus becomes the recovery checklist.

Visual placeholder

Visual placeholder: the Sankey plus the results summary; it turns a large test set into a manageable map.

Development: failure taxonomy

exit 255: missing device-side code objects.

invalid device function: target mismatch.

Missing TensileLibrary.dat: missing matrix artefacts.

Visual placeholder

Visual placeholder: a taxonomy table; it helps the audience sort the errors before they hear the fixes.

Development: source build setup

Match rocm-examples to ROCm 6.4.4.

Satisfy missing development dependencies.

Point the build to the actual host stack.

Visual placeholder

Visual placeholder: the git checkout and CMake configure log; it shows the build moving from package to source.

Development: assembly_to_executable port

main_gfx90c.s was missing even though hipcc could compile the HIP source.

The generated assembly is target-specific.

The fix points to build-system support, not manual editing.

Visual placeholder

Visual placeholder: the missing-file error plus the generated assembly list; it exposes the exact bottleneck.

Development: compiler pipeline

HIP -> LLVM IR -> AMDGPU machine code.

The pipeline explains why support can exist in LLVM but not in packaged ROCm.

This is the key bridge from theory to implementation.

Visual placeholder

Visual placeholder: a layered compiler pipeline diagram; it links the blog narrative to the actual toolchain.

Development: execution model

Work-item, wavefront, workgroup, dispatch.

VALU and SALU split vector and scalar paths.

The model explains occupancy and scheduling constraints.

Visual placeholder

Visual placeholder: the execution-hierarchy diagram; it makes the terminology usable in the later debugging slides.

Development: memory model

Registers, LDS, GDS, caches, VRAM, scratch.

gfx90c's memory hierarchy is small compared with discrete Vega assumptions.

The mismatch helps explain spoofing limits.

Visual placeholder

Visual placeholder: the memory-hierarchy diagram; it shows why the same ISA can still behave differently.

Development: gfx900 versus gfx90c

Both are Vega-family targets, but the silicon roles differ.

gfx900 assumes a discrete HBM2 GPU.

gfx90c is a 7-CU APU with DDR4 and 1 MB L2.

Visual placeholder

Visual placeholder: a side-by-side hardware comparison; it makes the spoofing discussion concrete.

Testing & debugging: what worked

Simple HIP kernels run successfully on native gfx90c.

Source-built paths recover some failing examples.

The build system can be nudged toward support without spoofing everywhere.

Visual placeholder

Visual placeholder: a pass list with green checkmarks; it balances the failure-heavy slides with proven wins.

Testing & debugging: what still fails

Advanced rocBLAS paths still need target artefacts.

Some benchmarks remain blocked by memory-coherency assumptions.

A small subset fails for genuine hardware reasons.

Visual placeholder

Visual placeholder: a red and amber issue board; it keeps unresolved items separate from the solved ones.

Project outcome: July summary

The problem is now decomposed into recoverable layers.
Source builds and assembly generation became concrete.
The path to gfx90c support is narrower and better defined.

Visual placeholder

Visual placeholder: a summary slide with three outcome tiles; it gives the report a clean checkpoint.

Project outcome: next steps

Finish the remaining rocm-examples porting work.

Generate or backport the missing Tensile data.

Extend the same workflow to gfx902 and beyond.

Visual placeholder

Visual placeholder: a next-steps roadmap; it keeps the report future-facing without overloading the conclusion.

Conclusion

The project is moving from workaround to method.

Software gaps dominate the failure surface.

The goal is a reusable recovery pipeline, not a one-off fix.

Visual placeholder

Visual placeholder: a closing funnel or staircase; it makes the final message easy to retain.

References

Blogs

- #1 framing
- #2 native ROCm
- #3 spoofing
- #4 AMD benchmarks
- #5 GPU basics
- #6 rocm-examples
- #7 source build
- #8 Vega model

Official cross-checks

- HIP model and SAXPY
- GPU patterns
- HIPIFY
- rocm-examples
- TheRock
- Tensile
- AMD ISA docs
- AMDGPU backend refs

Report trail

15-07-2026 and 31-07-2026.